STRATA: A Budgeted, Stack-Disciplined Scoped Symbol Table Evaluated on Compiler-Generated Traces

Shubhi Singh and Adhyan Jain
School of Computer Science and Engineering
Vellore Institute of Technology, Vellore, India

***Abstract*—A compiler that must itself run in little memory needs a symbol table whose size can be bounded in advance. Our earlier table, METIS-X, stored one 32-byte slot per declaration and one heap block per name longer than 12 bytes, and an audit found that it refuses declarations when one name is shadowed about 250 levels deep. We rebuilt the table around a different key. STRATA interns each visible name once, keeps the innermost declaration in the name's record, saves hidden declarations on an undo stack, and places everything in one contiguous buffer that the caller may supply. Scopes are last-in first-out, so leaving a scope pops two stacks and empties the newest index slots; no tombstones or shifts are needed. The algorithm is the textbook undo-stack symbol table; what is specific here is the single budgeted buffer with a defined failure result. We evaluate ten tables on the symbol-table calls that an instrumented Tiny C Compiler makes while compiling 8 open-source projects. STRATA agrees with a reference model on all traces, has the lowest peak heap on all of them (23–76% below METIS-X), makes 98.7–99.9% fewer allocator calls, and replays the traces 6–49% faster than METIS-X, although on 3 traces the confidence intervals overlap. In fixed-buffer mode it needs 27–36 bytes per live declaration. It does not improve lookup latency, its scope exit is slower than METIS-X's, and an interning flat hash map is faster on most traces while using more memory. The table was not integrated into a compiler and was not run on a microcontroller.**

***Index Terms*—symbol table, lexical scope, embedded compilers, memory budget, open addressing, trace replay**

I. Introduction

A scoped symbol table maps identifiers to declarations and forgets the declarations of a block when the block ends [1]. On a hosted compiler its memory is negligible. On a compiler that runs on the target itself, or inside a tool with a fixed memory budget, three properties matter that general-purpose containers do not offer: a bound on memory that can be fixed before compilation starts, no allocator calls during compilation, and a defined result when the bound is reached.

This paper continues the METIS-X project [2]. METIS-X was a flat Robin Hood table [3] with one slot per declaration and short names stored in the slot. Its authors closed the project with three findings of their own: every component was known, the traces had been extracted from source text with regular expressions, and a string-keyed table has no place in a compiler that interns identifiers first, as the Tiny C Compiler (TCC) [4] does.

We take those findings as the starting point and contribute the following.

1) A source-level audit of METIS-X (Section II) with one reproduced correctness defect.
2) STRATA (Section III), a table that is itself the interner and the scope mechanism, in one buffer with a hard budget.
3) Traces of the real symbol-table calls of TCC on 8 projects, replacing the heuristic traces (Section IV).
4) A comparison of ten tables with all results reported, including those where STRATA loses (Section V).

We do not claim a new algorithm. Section VI names the source of each mechanism.

II. Audit of the Previous Implementation

We read METIS-X at commit `7b3dacf`. The findings that shaped the redesign are these.

Hard failure under deep shadowing. All declarations of one name share a hash value and therefore one probe cluster. The probe distance is an 8-bit field capped at 250, and an insertion that would exceed it is refused. On a trace that declares one name in 400 nested scopes, METIS-X refused 149 declarations and returned 298 wrong lookup results; no other table failed.

Lookup cost grows with shadowing. The lookup compares every live declaration of the name and keeps the deepest.

Allocation per long name. Names over 12 bytes are heap blocks. The repository's claim that 93.4% of names fit in 12 bytes is not supported by its own traces.

Rehash is not exception-safe, the slot array never shrinks, and each scope owns a heap vector. These three were established by reading the code.

Measurement. The published heap figures included a set of all names built by the harness inside the measurement window; the main baseline never reclaimed names; latency was measured per call with a clock about as expensive as the call.

III. Design

A. Layout

STRATA keeps all state in one array of 32-bit words (Fig. 1). The *index* is an open-addressing array with linear probing; a slot holds an 8-bit fingerprint and a 24-bit reference

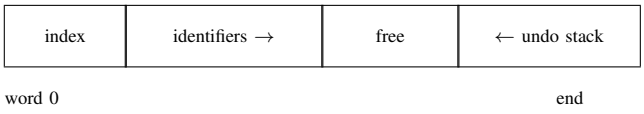


Fig. 1. The single buffer. References to identifier records are relative to the start of their region and references into the undo stack are relative to the end of the buffer, so either boundary can move without patching stored references.

to an identifier record. The *identifier stack* grows upward and holds one record per distinct visible name: a value word, a meta word (name length in words, an 8-bit type tag, the scope depth of the innermost declaration) and the name. The *undo stack* grows downward from the end of the buffer.

B. Operations

Lookup. The query is packed, hashed and probed; a slot is examined further only if its fingerprint matches. The record found holds the innermost declaration, so the cost does not depend on shadowing depth.

Declaration. If the name is not present, a record is appended, an index slot is filled and one word (the record's reference) is pushed on the undo stack. If the name is present with the current depth, the declaration is replaced and the call reports a redeclaration. Otherwise the record's current value and tag are pushed on the undo stack (three words) and overwritten. All space is reserved before anything is written, so a refused declaration changes nothing.

Scope entry pushes a two-word mark holding the identifier-stack top.

Scope exit pops undo entries down to the mark, restoring hidden declarations, then removes the names first seen in the scope. Those names are the most recently inserted keys of the index. In linear probing a key is placed in the first free slot of its probe path, so no older key's path depends on a newer key, and the newest keys can be removed by emptying their slots. No tombstones accumulate and nothing is shifted. Two passes are used: the slots are located first and emptied afterwards, because emptying one could cut the path to another.

C. Names

Characters of C identifiers are packed five to a word, six bits each; names containing other bytes are stored unpacked with a flag. The query is packed once and compared word by word, so stored names are never unpacked. The idea follows DEC's RADIX-50 encoding [5].

D. Memory modes

In *fixed* mode the caller supplies the buffer and the table never allocates. When the buffer is full, declarations and scope entries return an out-of-memory status. The index starts at 16 slots and doubles in place: the identifier stack is moved up and the index is rebuilt by re-inserting the records oldest first, which preserves the removal property above. In *growable* mode the buffer grows by half through `realloc`, up to an optional budget, and is halved when less than a quarter is in use. A failed allocation leaves the table unchanged. No operation throws.

TABLE I
COMPILER TRACES

Trace	Units	Events	Declare	Lookup	Names	Peak live	Depth
cjson	2	20,874	5,247	9,194	1,366	1,067	10
freertos	9	37,875	10,346	17,417	2,226	1,505	16
littlefs	2	21,080	3,835	12,230	1,001	732	16
lua	34	188,509	58,619	86,887	3,622	1,365	14
mbedtls	105	983,008	319,886	438,876	7,622	2,302	14
sqlite	1	268,960	27,072	172,426	7,801	4,245	22
tinycc	1	89,135	13,776	54,623	6,704	5,509	27
zlib	15	58,058	16,341	27,405	1,041	571	23

E. Limits

Names of 1–255 bytes, scope depth 16,383, buffer 64 MiB. A rebuild is linear in the number of live names, so worst-case latency of a declaration is not constant. The hash (MurmurHash3's 32-bit mixer [6] over packed words) is unkeyed.

IV. Methodology

A. Compiler traces

We added logging to TCC at commit `43c7708` where it touches its symbol tables: `sym_find` (lookup), `sym_link` and `global_identifier_push` (declaration of ordinary identifiers), and every change of its scope level, including prototype parameter scopes. The patched compiler compiled cJSON 1.7.18, littlefs 2.9.3, zlib 1.3.1, Lua 5.4.7, FreeRTOS-Kernel 11.1.0 (POSIX port), Mbed TLS 3.6.2 (three files that TCC rejects are missing), TinyCC itself and the SQLite 3.46.0 amalgamation. Table I describes the traces. A translation unit starts with a fresh table. Struct tags, members, labels and macros are in other TCC tables and are not traced.

B. Tables compared

Three configurations of STRATA; METIS-X and its two baselines, unmodified; and four tables written for this study on standard containers: a hash map per open scope; the undo-log algorithm of LLVM's `ScopedHashTable` [7] on `std::unordered_map` and on a Swiss-table node map [8]; and an interning Swiss flat map with dense per-identifier heads, which mirrors the split between interning and resolution in Clang [9]. All tables implement the same operations with the same semantics, and names are passed as existing string objects.

C. Measurements

Each table and trace gets four passes. (i) Every result is compared with a reference model built on `std::map`. (ii) Heap is measured through an allocator hook using `malloc_usable_size`; a `realloc` that moves a block is charged for both blocks. (iii) The whole trace is replayed 15 times after two warm-up replays, with one timer per replay; we report the median and a 95% bootstrap interval. (iv) Per-call latency is taken with the time-stamp counter, with the

TABLE II
PEAK HEAP IN KIB (LOWEST IN BOLD)

Trace	STRATA	METIS-X	map/scope	std+undo	Swiss+undo	Swiss int.
cjson	**43**	103	72	116	110	140
freertos	**58**	217	116	147	129	293
littlefs	**26**	107	54	68	57	142
lua	**64**	107	97	128	115	142
mbedtls	**144**	234	189	278	269	311
sqlite	**195**	451	333	513	494	1125
tinycc	**324**	423	425	551	449	576
zlib	**17**	53	44	60	55	71

TABLE III
SMALLEST FIXED BUFFER, AND REDUCTION (%) AGAINST THE PEAK HEAP OF FOUR TABLES

Trace	KiB	B/decl	METIS-X	map/scope	std+undo	Swiss int.
cjson	30.0	29	71	58	74	79
freertos	44.0	30	80	62	70	85
littlefs	20.0	28	81	63	71	86
lua	35.9	27	66	63	72	75
mbedtls	80.0	36	66	58	71	74
sqlite	140.4	34	69	58	73	88
tinycc	156.5	29	63	63	72	73
zlib	15.8	28	70	65	74	78

TABLE IV
REPLAY TIME, NS PER TRACE OPERATION (LOWEST IN BOLD)

Trace	STRATA	METIS-X	map/scope	std+undo	Swiss+undo	Swiss int.
cjson	32.2	36.2	41.6	35.1	35.4	**26.3**
freertos	**39.2**	76.5	49.0	41.9	64.3	49.3
littlefs	29.8	34.8	51.6	31.2	32.4	**23.4**
lua	38.7	50.0	53.1	44.9	48.7	**34.3**
mbedtls	46.7	62.0	56.1	57.3	56.9	**39.2**
sqlite	30.0	36.9	61.6	29.6	31.8	**28.1**
tinycc	33.6	35.6	59.3	42.0	32.4	**25.2**
zlib	27.8	36.5	46.7	40.2	38.2	**24.8**

TABLE V
REDUCTION (%) OF STRATA AGAINST METIS-X; NEGATIVE MEANS STRATA IS WORSE. CI: WHETHER THE 95% INTERVALS OF THE REPLAY MEDIANS OVERLAP. LOOKUP, DECLARE, EXIT: P95 LATENCY.

Trace	Heap	Allocs	Replay	CI	Lookup	Declare	Exit
cjson	58	99.4	11	overlap	20	52	-45
freertos	73	99.0	49	apart	15	63	-21
littlefs	76	99.3	14	apart	-17	53	-13
lua	40	99.2	23	apart	-44	42	-54
mbedtls	38	99.5	25	apart	3	50	-55
sqlite	57	99.9	19	overlap	11	57	-36
tinycc	23	99.8	6	overlap	-20	20	-57
zlib	67	98.7	24	apart	-4	54	-50

measured cost of an empty interval (19 ns) subtracted; these values are near the method's resolution and are secondary. For fixed mode we find by bisection the smallest buffer that replays a trace without a refused operation. Runs were pinned to one core of an Intel Core Ultra 7 258V under Linux (WSL 2), GCC 11.4, `-O2`.

V. RESULTS

A. Correctness

STRATA passes its unit tests under AddressSanitizer and UndefinedBehaviorSanitizer, including randomised sequences against the reference model, fixed-buffer exhaustion, and an allocator that fails after k calls. In the benchmark, 88 of 88 table–trace pairs on the compiler traces had no wrong or refused result.

B. Memory

Table II gives peak heap. STRATA is lowest on every trace: 23–76% below METIS-X (median 58%) and 44–83% below the interning flat map. It makes 98.7–99.9% fewer allocator calls than METIS-X. At the memory peak the number of live declarations equals the number of distinct live names on all traces but one, so the saving does not come from storing shadowed names once; it comes from the smaller record and from not allocating per name.

In fixed mode (Table III) the smallest sufficient buffer is 15.8–156.5 KiB, or 27–36 bytes per live declaration. That is 63–81% below METIS-X's peak heap and 73–88% below the interning flat map's. The 85% reduction we had set as a target is reached only against the latter, on 2 traces.

C. Time

Table IV gives replay time and Table V the comparison with METIS-X. STRATA replays every trace faster than METIS-X, by 6–49% (median 21%), but on 3 traces (cjson, sqlite, tinycc) the intervals overlap. The interning flat map is faster than STRATA on 7 of 8 traces, by a median of 16%: it never removes a name within a unit and does almost nothing at scope exit.

Per call, STRATA's declaration p95 is 20–63% below METIS-X's. Lookup p95 changes by between -44% and 20% with both signs: lookup is not improved. Scope-exit p95 is 13–57% *higher* than METIS-X's, because each name that dies is re-hashed to find its slot. The interquartile range of the replay times has a median of 14% of the median and reaches 54%; a second complete run moved individual timings by up to tens of percent while leaving the memory results unchanged.

D. Options and stress cases

Name packing reduces the minimum buffer by 5.5–9.0% and is slower on 5 of 8 traces (by up to 10%). Limiting the index to half full instead of three quarters lowers replay time by a median of 13% and enlarges the minimum buffer by 18–27%. On a synthetic trace that enters a 10,000-declaration scope 20 times, STRATA is slower than METIS-X (138 against 106 ns per operation) because it returns the buffer after each scope and regrows it; with shrinking disabled it takes 73 ns. With every key forced into one probe cluster, a lookup among 4,096 names takes 1744 ns instead of 14 ns: the table has no defence against adversarial keys. On the four heuristic traces of the earlier study, kept as a supplement, peak heap is 51–75% and replay time 10–41% below METIS-X.

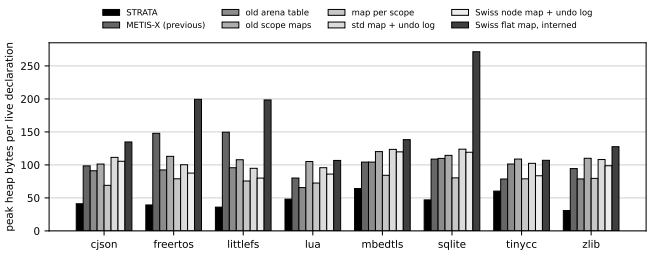


Fig. 2. Peak heap per live declaration on the compiler traces.

VI. Related Work

The algorithm of STRATA, a single hash table with an auxiliary stack that is unwound at scope exit, is described in compiler textbooks [1], [10] and implemented in LLVM's `ScopedHashTable` [7]. Clang separates an identifier table from a resolver that chains declarations per identifier [9]; TCC keeps a per-token symbol pointer and pops symbol stacks [4]. Fingerprints stored beside the key follow Swiss tables [11]. Deletion in linear probing normally needs tombstones or shifting [12], [13]; last-in first-out deletion needs neither, which follows directly from the insertion rule. Character packing goes back to RADIX-50 [5], and stack-like allocation by lifetime to arena allocators in compilers [14]. Relative to these, the specific points of this work are the single caller-supplied buffer with a defined failure result, the compiler-generated traces, and the measurements. Our search of prior work was not systematic.

VII. Limitations

STRATA was not integrated into a compiler; replaying TCC's calls measures the table, not a compilation, and symbol tables are a small part of compile time and memory. Nothing was run on a microcontroller. The traces reflect one compiler and C only. Timings come from one laptop under a hypervisor and vary between runs. Four of the baselines were written by us after known designs and are not the production code of LLVM or Clang. The exception-safety defect of METIS-X was found by inspection, not triggered.

VIII. Conclusion

Rebuilding the table around the identifier instead of the declaration removed the defects found in METIS-X and gave the lowest memory of ten tables on compiler-generated traces, with a fixed-buffer mode whose required size we can state per project. It did not give faster lookups, it made scope exit slower, and a simpler interning table is faster when memory is not constrained. The next steps are to replace TCC's own tables with STRATA and measure compilation, and to run the fixed-buffer mode on a microcontroller.

Acknowledgment

The audit, implementation, experiments and this draft were produced with an AI coding assistant (Claude, Anthropic) working in the first author's environment. The authors are responsible for verifying the code and the claims before any submission.

Data and Code Availability

Source code, traces, raw measurements (run `20261011-161306-final`) and the scripts that generate every table and figure are in the project repository. METIS-X and the Swiss-table library are fetched at pinned versions and are not redistributed.

References

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.

[2] A. Jain and S. Singh, "METIS: symbol-table research repository, commit 7b3dacf," https://github.com/adhyan-jain/Metis, 2026.

[3] P. Celis, P.-Å. Larson, and J. I. Munro, "Robin Hood hashing," in *Proc. 26th Annual Symposium on Foundations of Computer Science (FOCS)*, 1985.

[4] F. Bellard and TinyCC contributors, "Tiny C Compiler," https://repo.or.cz/tinycc.git, commit 43c7708.

[5] Wikipedia contributors, "DEC RADIX 50," https://en.wikipedia.org/wiki/DEC_RADIX_50.

[6] A. Appleby, "MurmurHash3, SMHasher," https://github.com/aappleby/smhasher.

[7] LLVM Project, "ScopedHashTable.h, LLVM ADT library," https://llvm.org/doxygen/ScopedHashTable_8h_source.html.

[8] G. Popovitch, "The parallel hashmap, version 1.4.1," https://github.com/greg7mdp/parallel-hashmap.

[9] LLVM Project, "IdentifierTable.cpp and IdentifierResolver.h, Clang," https://github.com/llvm/llvm-project/tree/main/clang.

[10] A. W. Appel, *Modern Compiler Implementation in C*. Cambridge University Press, 1998.

[11] Abseil, "Swiss tables design notes," https://abseil.io/about/design/swisstables.

[12] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.

[13] M. A. Bender, B. C. Kuszmaul, and W. Kuszmaul, "Linear probing revisited: Tombstones mark the demise of primary clustering," in *Proc. 62nd IEEE Annual Symposium on Foundations of Computer Science (FOCS)*, 2021, pp. 1171–1182.

[14] D. R. Hanson, "Fast allocation and deallocation of memory based on object lifetimes," *Software: Practice and Experience*, vol. 20, no. 1, pp. 5–12, 1990.